

MSES602 - Lab #1: Introduction to DevOps - Traditional Ops with Python

Course: MSES602

Lab: #1 - Introduction to DevOps: Traditional Ops with Python

Instructor: Professor Shari Plantz-Masters, Ph.D.

Student: Abdullah Bahir

Date: August 30, 2026

Overview

This lab introduced traditional systems administration tasks performed manually and with Python scripting. The goal was to set up an Ubuntu 24.04.4 LTS Virtual Machine (VM name: MSE602-Ubuntu), configure SSH access, verify Python and Git installations, and execute several operational Python scripts. The lab reinforces the DevOps principle of automating repetitive tasks to reduce human error and increase efficiency--shifting from treating systems as "pets" to treating them as "cattle."

Section 1: VM Setup and SSH Configuration

Install and Verify SSH

After booting the Ubuntu VM (MSE602-Ubuntu), SSH was already at the newest version (1:9.6p1-3ubuntu13.18) and was verified to be running with the service ssh status command. The service showed as active (running) on host abdullah-pc.

```
sudo apt install ssh
service ssh status
```

Screenshot sc1 - SSH Service Status:

```
ssh is already the newest version (1:9.6p1-3ubuntu13.18).
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
abdullah@abdullah-pc:~$ service ssh status
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-08-30 12:22:34 CDT; 2h 4min ago
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 2915 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 2916 (sshd)
     Tasks: 1 (limit: 4234)
    Memory: 2.3M (peak: 6.7M)
       CPU: 59ms
   CGroup: /system.slice/ssh.service
           └─2916 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

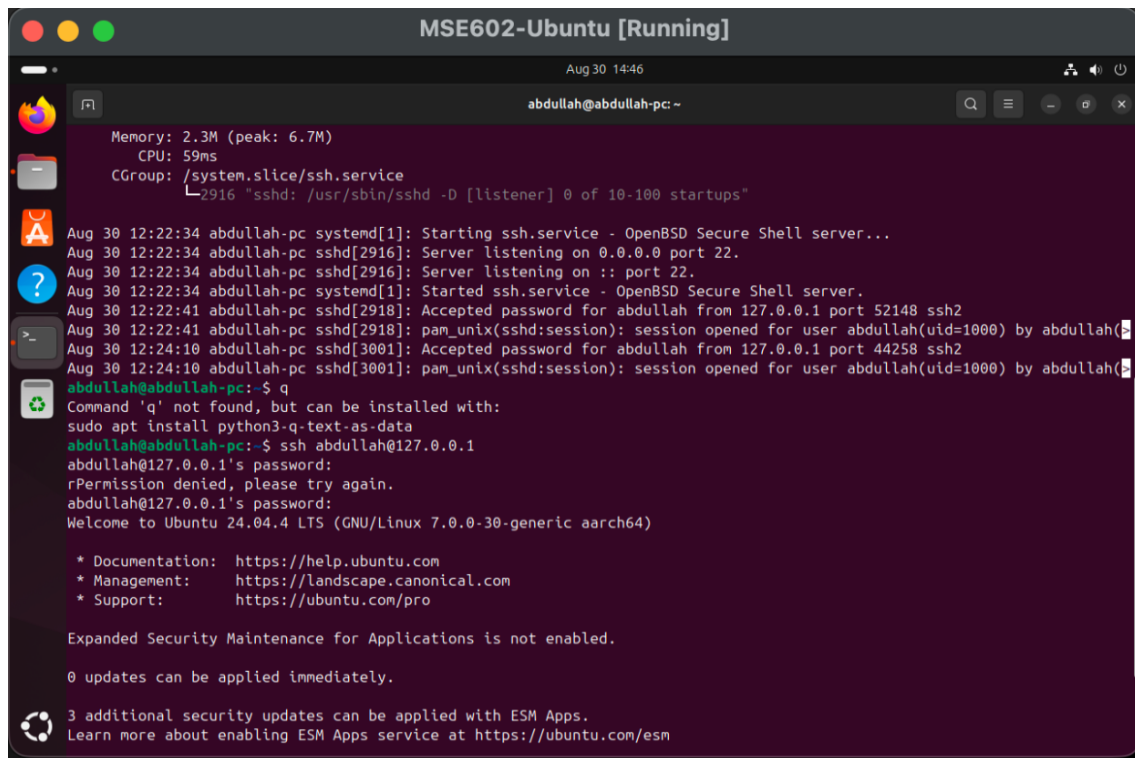
Aug 30 12:22:34 abdullah-pc systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Aug 30 12:22:34 abdullah-pc sshd[2916]: Server listening on 0.0.0.0 port 22.
Aug 30 12:22:34 abdullah-pc sshd[2916]: Server listening on :: port 22.
Aug 30 12:22:34 abdullah-pc systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Aug 30 12:22:41 abdullah-pc sshd[2918]: Accepted password for abdullah from 127.0.0.1 port 52148 ssh2
Aug 30 12:22:41 abdullah-pc sshd[2918]: pam_unix(sshd:session): session opened for user abdullah(uid=1000) by abdullah(
Aug 30 12:24:10 abdullah-pc sshd[3001]: Accepted password for abdullah from 127.0.0.1 port 44258 ssh2
Aug 30 12:24:10 abdullah-pc sshd[3001]: pam_unix(sshd:session): session opened for user abdullah(uid=1000) by abdullah(
abdullah@abdullah-pc:~$ q
Command 'q' not found, but can be installed with:
sudo apt install python3-q-text-as-data
abdullah@abdullah-pc:~$
```

Test SSH Loopback Connection

The SSH connection was tested locally using the loopback address 127.0.0.1 as user abdullah to confirm the service was accepting connections.

```
ssh abdullah@127.0.0.1
exit
```

Screenshot sc2 - Successful SSH Connection:



```
Memory: 2.3M (peak: 6.7M)
CPU: 59ms
CGroup: /system.slice/ssh.service
└─2916 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 30 12:22:34 abdullah-pc systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Aug 30 12:22:34 abdullah-pc sshd[2916]: Server listening on 0.0.0.0 port 22.
Aug 30 12:22:34 abdullah-pc sshd[2916]: Server listening on :: port 22.
Aug 30 12:22:34 abdullah-pc systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Aug 30 12:22:41 abdullah-pc sshd[2918]: Accepted password for abdullah from 127.0.0.1 port 52148 ssh2
Aug 30 12:22:41 abdullah-pc sshd[2918]: pam_unix(sshd:session): session opened for user abdullah(uid=1000) by abdullah
Aug 30 12:24:10 abdullah-pc sshd[3001]: Accepted password for abdullah from 127.0.0.1 port 44258 ssh2
Aug 30 12:24:10 abdullah-pc sshd[3001]: pam_unix(sshd:session): session opened for user abdullah(uid=1000) by abdullah

abdullah@abdullah-pc:~$ q
Command 'q' not found, but can be installed with:
sudo apt install python3-q-text-as-data
abdullah@abdullah-pc:~$ ssh abdullah@127.0.0.1
abdullah@127.0.0.1's password:
rPermission denied, please try again.
abdullah@127.0.0.1's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-30-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

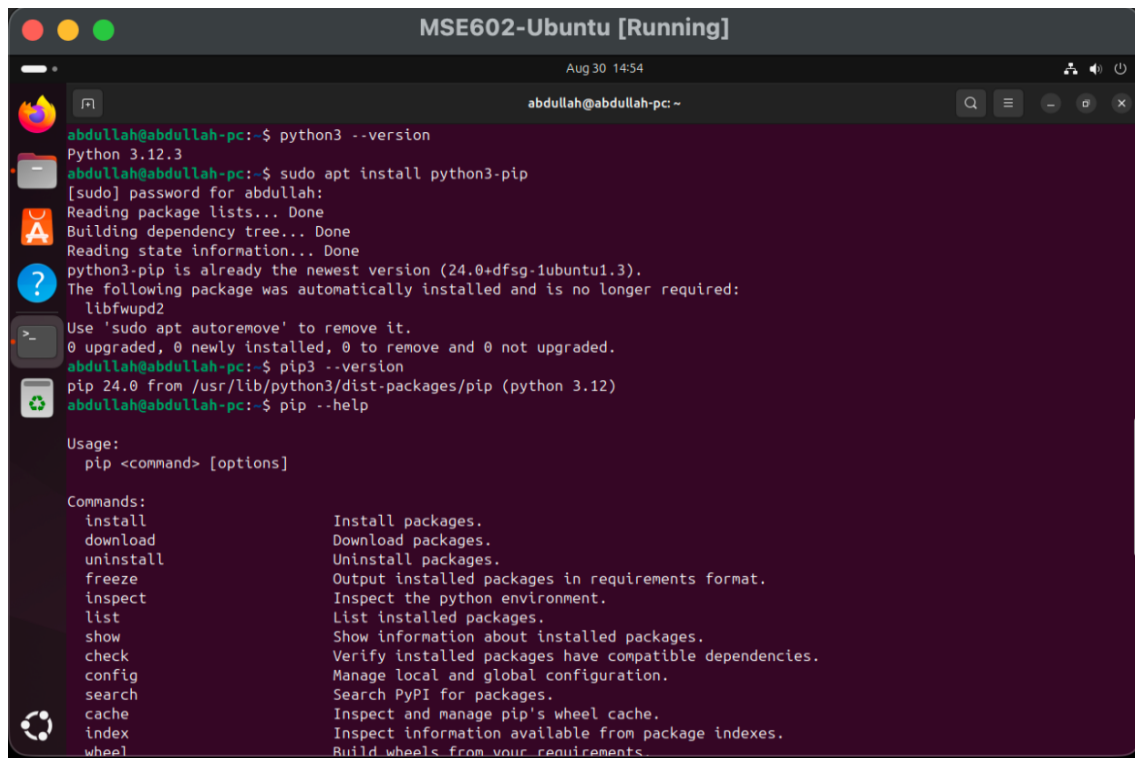
3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
```

Section 2: Python and Pip Verification

Python 3 was already installed on the Ubuntu VM. The version confirmed was Python 3.12.3. pip3 was already at the newest version (24.0+dfsg-1ubuntu1.3) and confirmed with pip3 --version showing pip 24.0.

```
python3 --version
sudo apt install python3-pip
pip3 --version
pip --help
```

Screenshot sc3 - Python3 Version and Pip Installation:



```
abdullah@abdullah-pc:~$ python3 --version
Python 3.12.3
abdullah@abdullah-pc:~$ sudo apt install python3-pip
[sudo] password for abdullah:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (24.0+dfsg-1ubuntu1.3).
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
abdullah@abdullah-pc:~$ pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
abdullah@abdullah-pc:~$ pip --help

Usage:
  pip <command> [options]

Commands:
  install           Install packages.
  download          Download packages.
  uninstall         Uninstall packages.
  freeze            Output installed packages in requirements format.
  inspect           Inspect the python environment.
  list              List installed packages.
  show              Show information about installed packages.
  check             Verify installed packages have compatible dependencies.
  config            Manage local and global configuration.
  search            Search PyPI for packages.
  cache             Inspect and manage pip's wheel cache.
  index             Inspect information available from package indexes.
  wheel             Build wheels from your requirements.
```

Section 3: Git Installation

Git was installed to enable cloning of remote repositories. Git was already at the newest version (1:2.43.0-1ubuntu7.3) and confirmed with `git --version` showing git version 2.43.0.

```
sudo apt install git
git --version
```

Screenshot sc4 - Git Version:

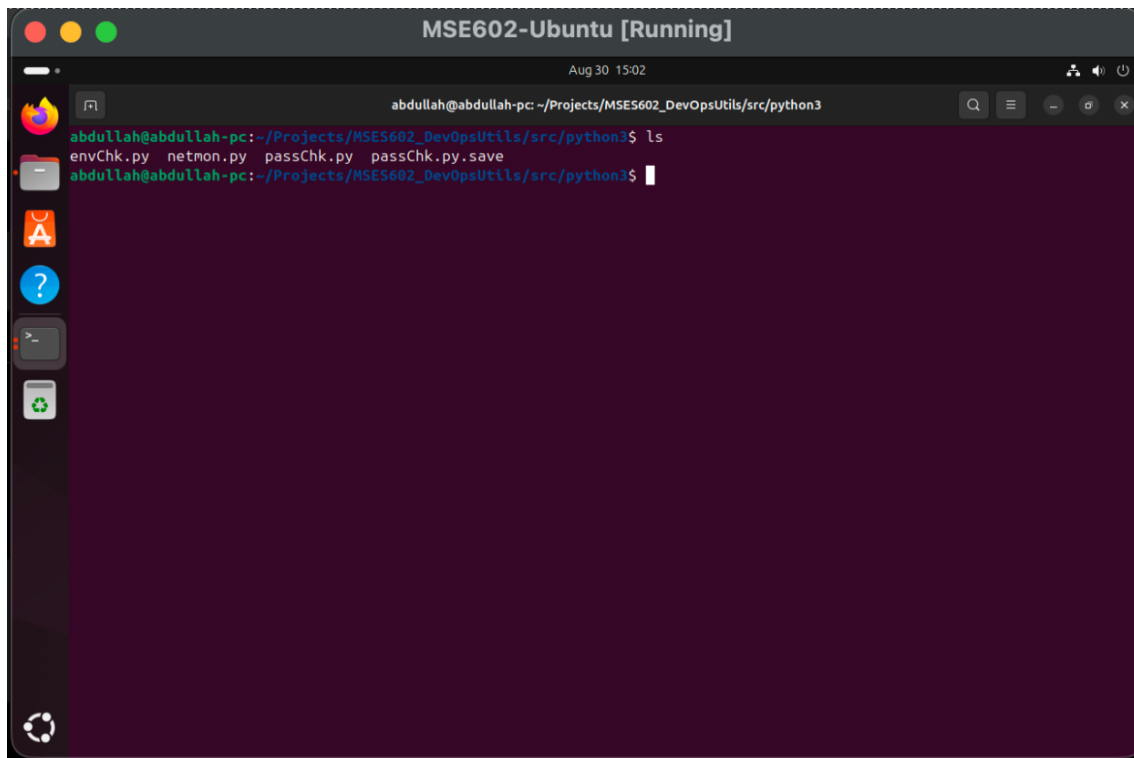
```
MSE602-Ubuntu [Running]
Aug 30 14:58
abdullah@abdullah-pc: ~
--exists-action <action>      Default action when a path already exists: (s)witch, (i)gnore, (w)ipe, (b)ackup,
                               (a)bort.
--trusted-host <hostname>     Mark this host or host:port pair as trusted, even though it does not have valid or any
                               HTTPS.
--cert <path>                 Path to PEM-encoded CA certificate bundle. If provided, overrides the default. See 'SSL
                               Certificate Verification' in pip documentation for more information.
--client-cert <path>          Path to SSL client certificate, a single file containing the private key and the
                               certificate in PEM format.
--cache-dir <dir>             Store the cache data in <dir>.
--no-cache-dir                Disable the cache.
--disable-pip-version-check    Don't periodically check PyPI to determine whether a new version of pip is available for
                               download. Implied with --no-index.
--no-color                    Suppress colored output.
--no-python-version-warning    Silence deprecation warnings for upcoming unsupported Pythons.
--use-feature <feature>       Enable new functionality, that may be backward incompatible.
--use-deprecated <feature>    Enable deprecated functionality, that will be removed in the future.
abdullah@abdullah-pc:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
abdullah@abdullah-pc:~$ y
y: command not found
abdullah@abdullah-pc:~$ git --version
git version 2.43.0
abdullah@abdullah-pc:~$
```

Section 4: Clone the Project Repository

A Projects directory was created and the MSES602 DevOps utilities repository was cloned from GitHub. The Python scripts were located at ~/Projects/MSES602_DevOpsUtils/src/python3 and included: envChk.py, netmon.py, passChk.py, and passChk.py.save.

```
cd
mkdir Projects
cd Projects
git clone https://github.com/RegisUniversity/MSES602_DevOpsUtils.git
cd ./MSES602_DevOpsUtils/src/python3
ls
```

Screenshot sc5 - Cloned Repository and Python Files:

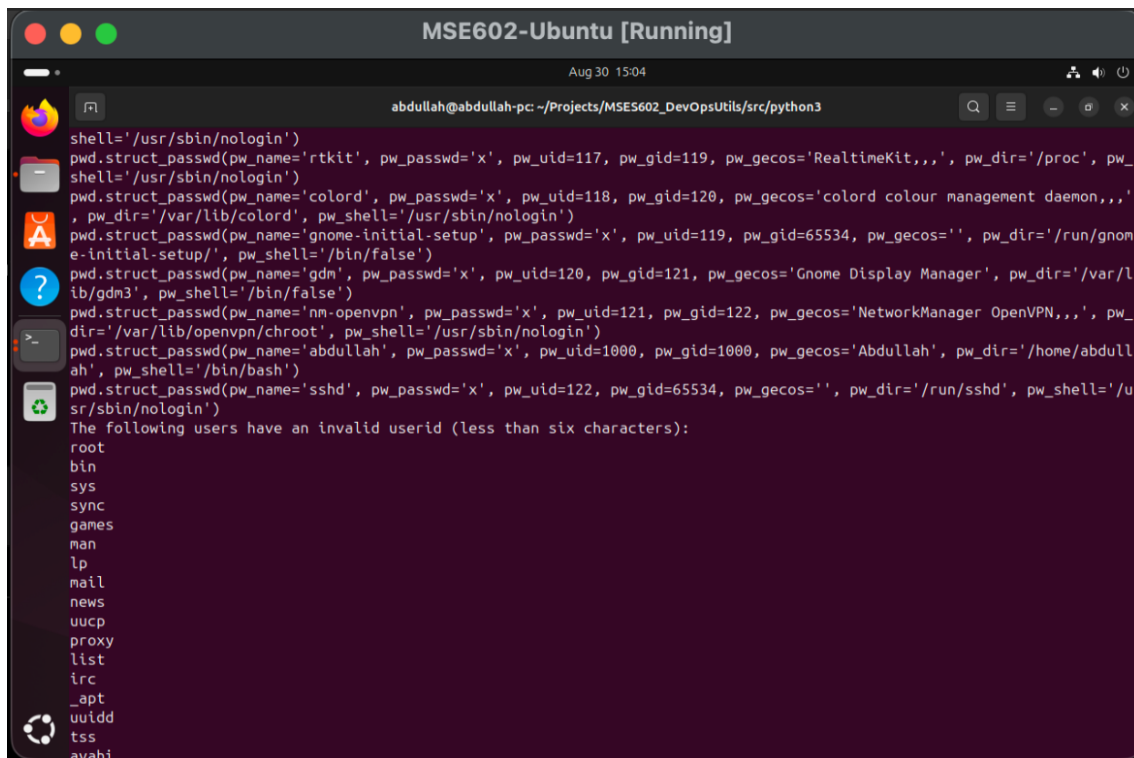


Section 5: Running passChk.py - Password Security Audit

The passChk.py script scans all user accounts on the Ubuntu image and flags any accounts with short usernames or passwords that would be easily guessed by brute-force tools.

```
python3 passChk.py
```

Screenshot sc6 - passChk.py Output:



```
MSE602-Ubuntu [Running]
Aug 30 15:04
abdullah@abdullah-pc: ~/Projects/MSE602_DevOpsUtils/src/python3

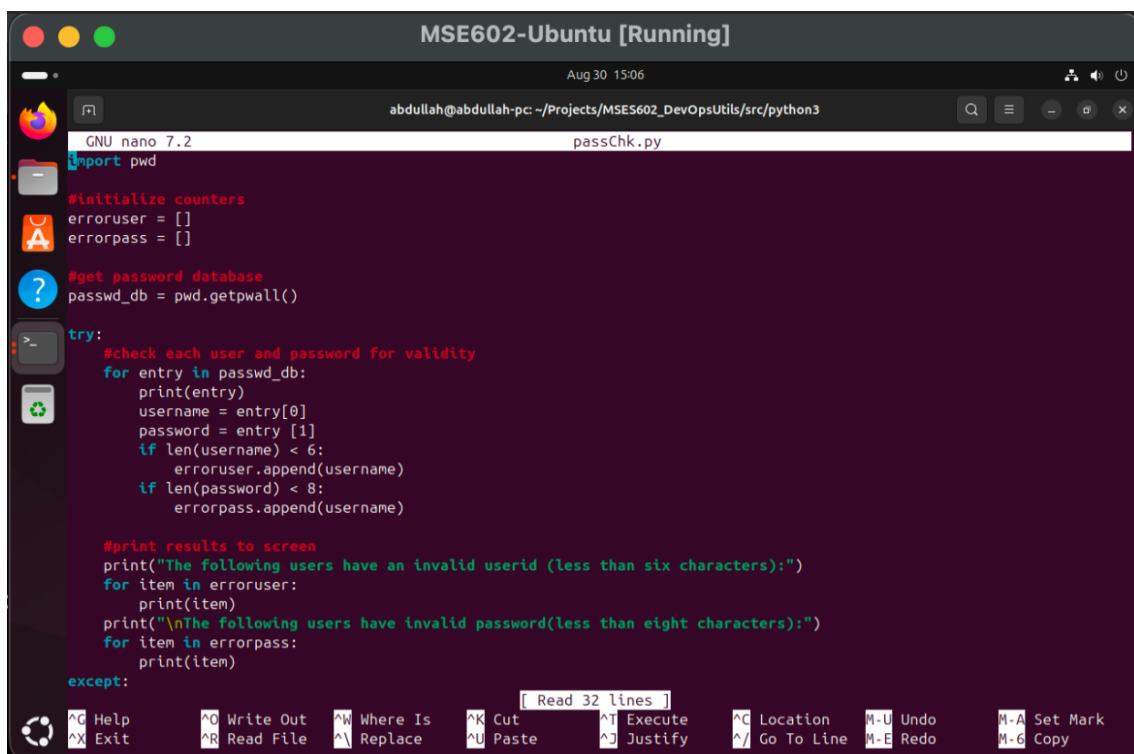
shell='/usr/sbin/nologin')
pwd.struct_passwd(pwd_name='rtkit', pw_passwd='x', pw_uid=117, pw_gid=119, pw_gecos='RealtimeKit,,,', pw_dir='/proc', pw_
shell='/usr/sbin/nologin')
pwd.struct_passwd(pwd_name='colord', pw_passwd='x', pw_uid=118, pw_gid=120, pw_gecos='colord colour management daemon,,,',
, pw_dir='/var/lib/colord', pw_shell='/usr/sbin/nologin')
pwd.struct_passwd(pwd_name='gnome-initial-setup', pw_passwd='x', pw_uid=119, pw_gid=65534, pw_gecos='', pw_dir='/run/gnom
e-initial-setup/', pw_shell='/bin/false')
pwd.struct_passwd(pwd_name='gdm', pw_passwd='x', pw_uid=120, pw_gid=121, pw_gecos='Gnome Display Manager', pw_dir='/var/l
ib/gdm3', pw_shell='/bin/false')
pwd.struct_passwd(pwd_name='nm-openvpn', pw_passwd='x', pw_uid=121, pw_gid=122, pw_gecos='NetworkManager OpenVPN,,,', pw_
dir='/var/lib/openvpn/chroot', pw_shell='/usr/sbin/nologin')
pwd.struct_passwd(pwd_name='abdullah', pw_passwd='x', pw_uid=1000, pw_gid=1000, pw_gecos='Abdullah', pw_dir='/home/abdull
ah', pw_shell='/bin/bash')
pwd.struct_passwd(pwd_name='sshd', pw_passwd='x', pw_uid=122, pw_gid=65534, pw_gecos='', pw_dir='/run/sshd', pw_shell='/u
sr/sbin/nologin')
The following users have an invalid userid (less than six characters):
root
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
list
irc
_apt
uuidd
tss
avahi
```

Reviewing the passChk.py Source Code

The script was opened in nano to review how it reads the system user and password files and applies its security checks.

```
nano passChk.py
```

Screenshot sc7 - passChk.py Source Code:



```
MSE602-Ubuntu [Running]
Aug 30 15:06
abdullah@abdullah-pc: ~/Projects/MSE602_DevOpsUtils/src/python3

GNU nano 7.2
passChk.py
import pwd

#initialize counters
erroruser = []
errorpass = []

#get password database
passwd_db = pwd.getpwall()

try:
    #check each user and password for validity
    for entry in passwd_db:
        print(entry)
        username = entry[0]
        password = entry[1]
        if len(username) < 6:
            erroruser.append(username)
        if len(password) < 8:
            errorpass.append(username)

    #print results to screen
    print("The following users have an invalid userid (less than six characters):")
    for item in erroruser:
        print(item)
    print("\nThe following users have invalid password(less than eight characters):")
    for item in errorpass:
        print(item)
except:
    pass

[ Read 32 lines ]
^O Help      ^O Write Out  ^W Where Is   ^R Cut        ^T Execute    ^C Location   ^U Undo       ^A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line  ^M Redo       ^M Copy
```

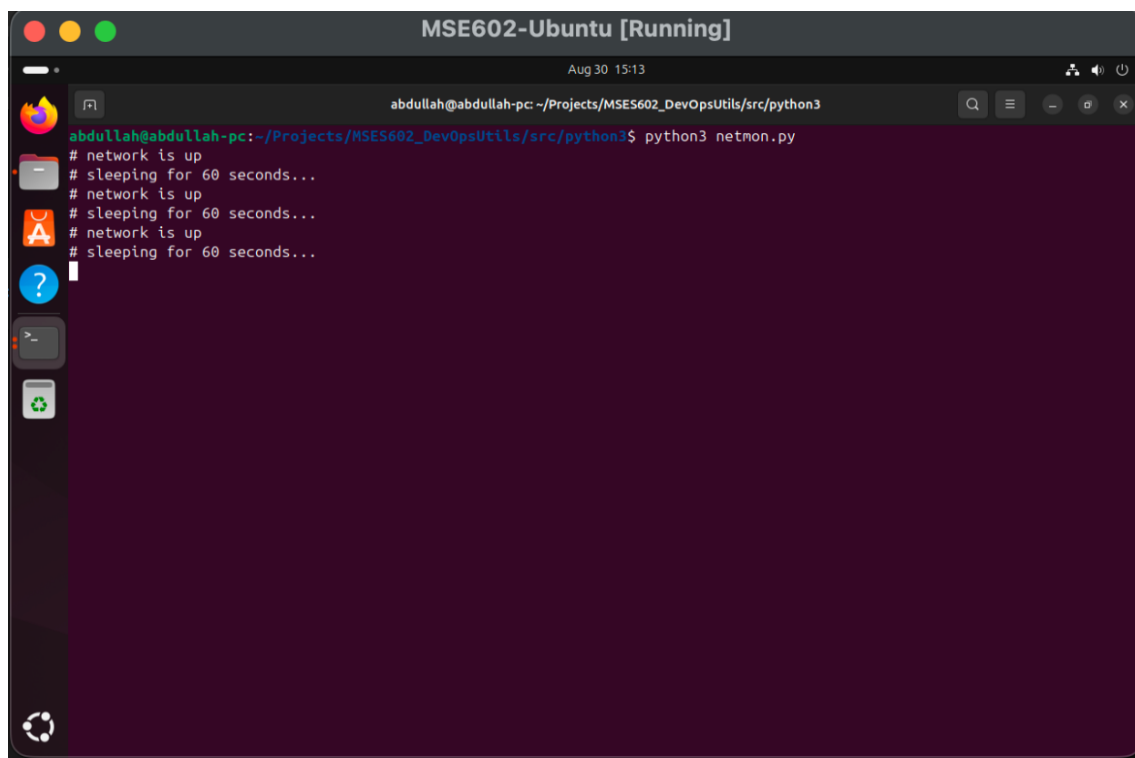
Observation: Most flagged accounts are system-level accounts (root, bin, sys, sync, games, man, lp, mail, etc.) with usernames shorter than six characters. The only active user account abdullah (uid=1000) passed the checks. Automating this check daily would be far more practical than manually reviewing the files each time.

Section 6: Running netmon.py - Network Monitor

The netmon.py script monitors network connectivity at 60-second intervals and reports whether the network is up or down. It can be extended to trigger alerts or corrective actions on failure.

```
python3 netmon.py
```

Screenshot sc8 - netmon.py Running:



Section 7: Extended envChk.py - System Environment Check

For the final task, the envChk.py script was extended with additional system metrics beyond the original. The new script (lab1.py) was written to include:

- System information (OS, hostname, machine)
- CPU usage (cores and current utilization)
- Memory usage (total, available, used)
- Disk usage (total, used, free)
- Network I/O counters
- SSH connectivity check - verifies port 22 is reachable on localhost
- System uptime since last boot

- Logged-in users
- Running process count
- Automated status warnings for CPU, memory, and disk thresholds

Extended Script: lab1.py

```

"""
MSES602 - Lab #1: Introduction to DevOps - Traditional Ops with Python
System environment check script (extends envChk.py with an SSH reachability check).
"""

import psutil
import platform
import socket
from datetime import datetime

def print_section(title):
    print()
    print("=" * 60)
    print(title)
    print("=" * 60)

print_section("SYSTEM INFORMATION")
print("Operating System:", platform.system())
print("OS Version:", platform.version())
print("Machine:", platform.machine())
print("Hostname:", socket.gethostname())

print_section("CPU INFORMATION")
print("CPU Cores:", psutil.cpu_count(logical=False))
print("Logical CPUs:", psutil.cpu_count(logical=True))
cpu_usage = psutil.cpu_percent(interval=1)
print("Current CPU Usage:", cpu_usage, "%")

print_section("MEMORY INFORMATION")
memory = psutil.virtual_memory()
print("Total Memory:", round(memory.total / (1024 ** 3), 2), "GB")
print("Available Memory:", round(memory.available / (1024 ** 3), 2), "GB")
print("Used Memory:", round(memory.used / (1024 ** 3), 2), "GB")
print("Memory Usage:", memory.percent, "%")

print_section("DISK INFORMATION")
disk = psutil.disk_usage("/")
print("Total Disk Space:", round(disk.total / (1024 ** 3), 2), "GB")
print("Used Disk Space:", round(disk.used / (1024 ** 3), 2), "GB")
print("Free Disk Space:", round(disk.free / (1024 ** 3), 2), "GB")
print("Disk Usage:", disk.percent, "%")

print_section("NETWORK INFORMATION")
network = psutil.net_io_counters()
print("Bytes Sent:", round(network.bytes_sent / (1024 ** 2), 2), "MB")
print("Bytes Received:", round(network.bytes_recv / (1024 ** 2), 2), "MB")

print_section("SSH CONNECTIVITY")

```

```

ssh_host = "127.0.0.1"
ssh_port = 22
try:
    with socket.create_connection((ssh_host, ssh_port), timeout=3):
        print(f"SSH port {ssh_port} on {ssh_host} is OPEN (reachable).")
except (socket.timeout, ConnectionRefusedError, OSError):
    print(f"SSH port {ssh_port} on {ssh_host} is NOT reachable.")

print_section("SYSTEM UPTIME")
boot_time = datetime.fromtimestamp(psutil.boot_time())
print("System Boot Time:", boot_time)
uptime_seconds = datetime.now().timestamp() - psutil.boot_time()
print("System Uptime:", round(uptime_seconds / 3600, 2), "hours")

print_section("LOGGED IN USERS")
users = psutil.users()
if users:
    for user in users:
        print("Username:", user.name)
        print("Terminal:", user.terminal)
        print("Login Time:", datetime.fromtimestamp(user.started))
else:
    print("No logged-in users found.")

print_section("RUNNING PROCESSES")
print("Number of Running Processes:", len(psutil.pids()))

print_section("SYSTEM STATUS")
print("WARNING: CPU usage is high." if cpu_usage > 80 else "CPU usage is normal.")
print("WARNING: Memory usage is high." if memory.percent > 80 else "Memory usage is normal.")
print("WARNING: Disk usage is high." if disk.percent > 80 else "Disk usage is normal.")

print()
print("System environment check completed.")

```

Install psutil and Run the Script

```

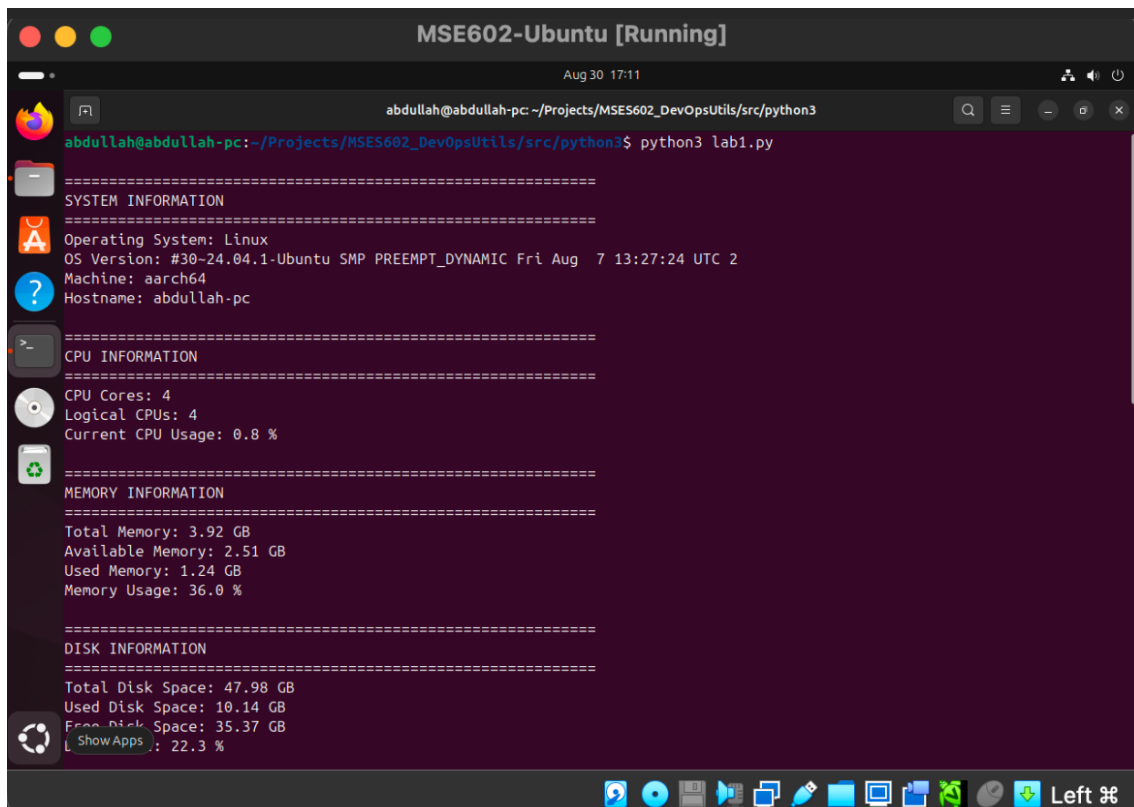
pip3 install psutil
python3 lab1.py

```

The script produced the following output on the MSE602-Ubuntu VM:

- OS: Linux, Ubuntu 24.04.4 LTS, aarch64, Hostname: abdullah-pc
- CPU: 4 cores / 4 logical CPUs, Usage: 0.8%
- Memory: 3.92 GB total, 1.24 GB used (36.0%)
- Disk: 47.98 GB total, 10.14 GB used (22.3%)
- Network: 0.06 MB sent, 0.08 MB received
- SSH: port 22 on 127.0.0.1 is OPEN (reachable)
- Boot Time: 2026-08-30 16:07:09, Uptime: 1.02 hours
- Logged-in user: abdullah (seat0 & tty2), Login: 2026-08-30 16:07:49
- Running Processes: 204
- Status: CPU, Memory, and Disk usage all normal

Screenshot sc9 - psutil Install and Script Output (Part 1):



The screenshot shows a terminal window titled "MSE602-Ubuntu [Running]" with a timestamp of "Aug 30 17:11". The user is logged in as "abdullah" at "abdullah-pc". The terminal shows the execution of a Python script "lab1.py" which outputs system information. The output is organized into sections: SYSTEM INFORMATION, CPU INFORMATION, MEMORY INFORMATION, and DISK INFORMATION. The system information includes the operating system (Linux), version (#30-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug 7 13:27:24 UTC 2), machine type (aarch64), and hostname (abdullah-pc). The CPU information shows 4 cores and 4 logical CPUs, with a current usage of 0.8%. The memory information shows a total of 3.92 GB, with 2.51 GB available and 1.24 GB used (36.0% usage). The disk information shows a total of 47.98 GB, with 10.14 GB used and 35.37 GB free (22.3% usage).

```
abdullah@abdullah-pc: ~/Projects/MSE602_DevOpsUtils/src/python3
python3 lab1.py

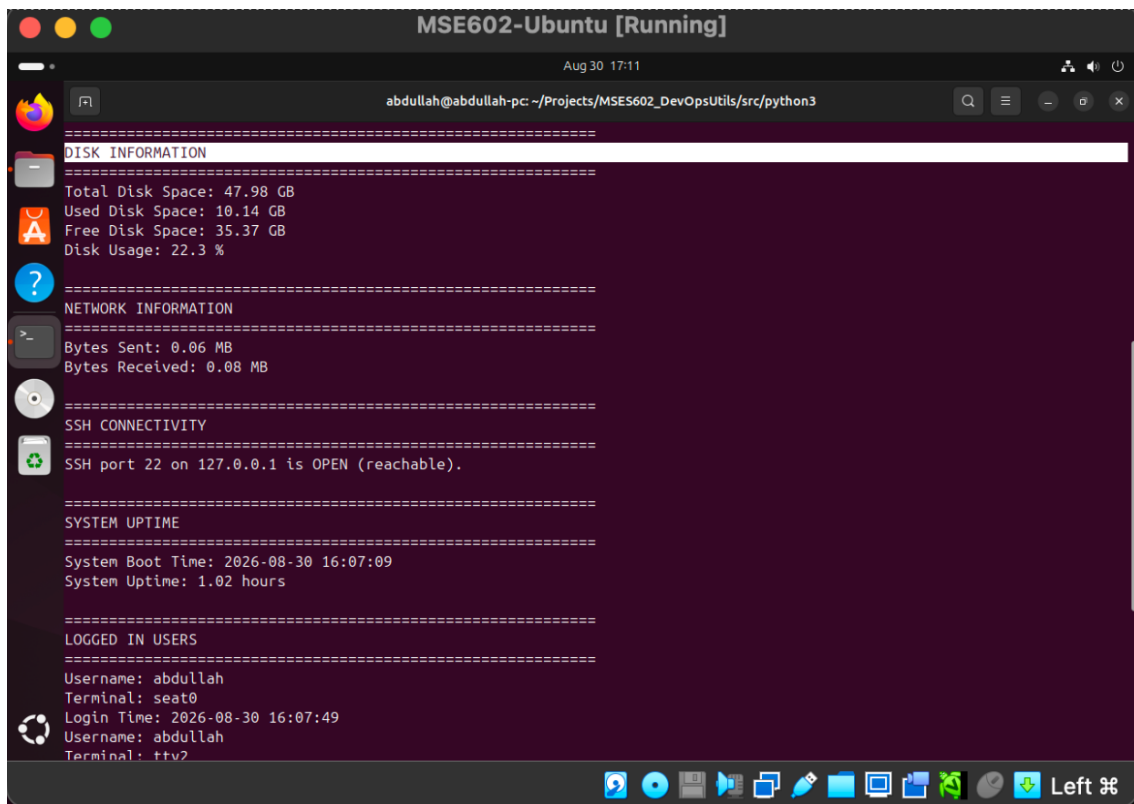
=====
SYSTEM INFORMATION
=====
Operating System: Linux
OS Version: #30-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Fri Aug 7 13:27:24 UTC 2
Machine: aarch64
Hostname: abdullah-pc

=====
CPU INFORMATION
=====
CPU Cores: 4
Logical CPUs: 4
Current CPU Usage: 0.8 %

=====
MEMORY INFORMATION
=====
Total Memory: 3.92 GB
Available Memory: 2.51 GB
Used Memory: 1.24 GB
Memory Usage: 36.0 %

=====
DISK INFORMATION
=====
Total Disk Space: 47.98 GB
Used Disk Space: 10.14 GB
Free Disk Space: 35.37 GB
Disk Usage: 22.3 %
```

Screenshot sc10 - Script Output (Part 2):



The screenshot shows the same terminal window as the previous one, but with the output of the Python script scrolled down. The output now shows the DISK INFORMATION section, followed by NETWORK INFORMATION, SSH CONNECTIVITY, SYSTEM UPTIME, and LOGGED IN USERS. The network information shows 0.06 MB sent and 0.08 MB received. The SSH connectivity check shows that port 22 on 127.0.0.1 is OPEN (reachable). The system uptime shows a boot time of 2026-08-30 16:07:09 and a system uptime of 1.02 hours. The logged in users section shows the current user as "abdullah" on terminal "seat0", with a login time of 2026-08-30 16:07:49.

```
=====
DISK INFORMATION
=====
Total Disk Space: 47.98 GB
Used Disk Space: 10.14 GB
Free Disk Space: 35.37 GB
Disk Usage: 22.3 %

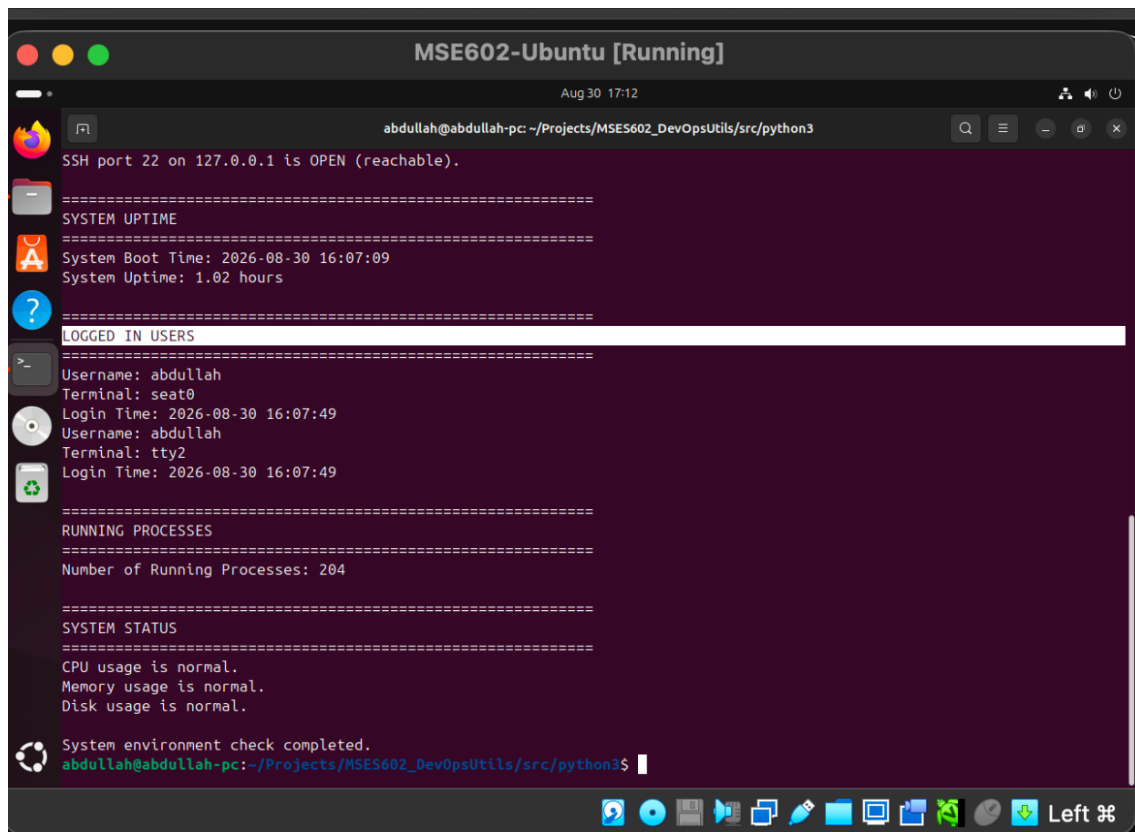
=====
NETWORK INFORMATION
=====
Bytes Sent: 0.06 MB
Bytes Received: 0.08 MB

=====
SSH CONNECTIVITY
=====
SSH port 22 on 127.0.0.1 is OPEN (reachable).

=====
SYSTEM UPTIME
=====
System Boot Time: 2026-08-30 16:07:09
System Uptime: 1.02 hours

=====
LOGGED IN USERS
=====
Username: abdullah
Terminal: seat0
Login Time: 2026-08-30 16:07:49
Username: abdullah
Terminal: ttv2
```

Screenshot sc11 - Script Output (Part 3 - SSH Check and Status):



Conclusion

This lab demonstrated traditional systems administration tasks on an Ubuntu VM, including SSH setup, Python scripting, and system monitoring. A key takeaway is that automating these tasks with Python eliminates manual effort, reduces errors, and enables repeatable, consistent system checks.

The extended lab1.py script goes beyond the original envChk.py by adding an SSH connectivity check, system uptime, logged-in users, process count, and threshold-based status warnings. These additions reflect practical, real-world operational monitoring concerns.

As emphasized in the DevOps Handbook, automation is central to the CAMS model (Culture, Automation, Measurement, Sharing). Scripts like these, stored and versioned in Git repositories, represent the shift from treating systems as "pets" to treating them as reproducible, disposable "cattle."

References

- Willis, J., Debois, P., Humble, J., & Kim, G. (2021). The DevOps Handbook, 2nd ed. IT Revolution Press.
- Linuxize (2018). How to install Pip on Ubuntu 18.04.
- Menchaca, J. (2018). DevOps Concepts: Pets vs Cattle.
- Parkash, A. (2018). Using apt-get Commands in Linux.
- Rendek, L. (2018). Enable SSH on Ubuntu 18.04 Bionic Beaver Linux.

- psutil documentation: <https://psutil.readthedocs.io/en/latest/>